ชื่อ_____ รหัสนักศึกษา_____ เลขที่นั่งสอบ_____

ข้อสอบครั้งที่ 2 วิชา 198322 สถาปัตยกรรมคอมพิวเตอร์ เทอมปลาย ปีการศึกษา 2561.
เวลาสอบ 3 ชั่วโมง.

คำสั่ง

- ไม่อนุญาตให้นำเอกสาร หนังสือ หรือตำราใดๆ เข้าสอบ
- ไม่อนุญาตการใช้โทรศัพท์หรือเครื่องมือสื่อสารใดๆ ยกเว้นเครื่องมือสำหรับส่งคำตอบ
- ไม่อนุญาตการใช้ระบบเครือข่ายสื่อสาร ยกเว้นการส่งคำตอบเข้าระบบรับคำตอบที่กำหนดไว้ (https://autolab.en.kku.ac.th)
- ไม่อนุญาตการสืบค้นข้อมูลจากระบบเครือข่าย
- อนุญาติให้ใช้เครื่องคิดเลข หรือโปรแกรมคำนวณได้ที่ติดตั้งอยู่แล้วในเครื่องที่ทำการสอบได้

จุดประสงค์

- วัดความเข้าใจพื้นฐานพีชคณิตของคอมพิวเตอร์
- วัดความเข้าใจพื้นฐานของฮาร์ดแวร์ที่ทำงานตาม machine code
- วัดความเข้าใจพื้นฐานของแนวคิด ประโยชน์ ความเสี่ยง การจัดการความเสี่ยงของ pipelining

---

คำถาม 1 (10 points)

แสดงการคำนวณพีชคณิตของคอมพิวเตอร์. กำหนดให้เป็น ระบบ 4 บิต.
ตอบเป็นฐานสอง
(คำถามนี้ทุกข้อร่วมกันเพื่อวัดจุดประสงค์เดียว ไม่มี partial credit)

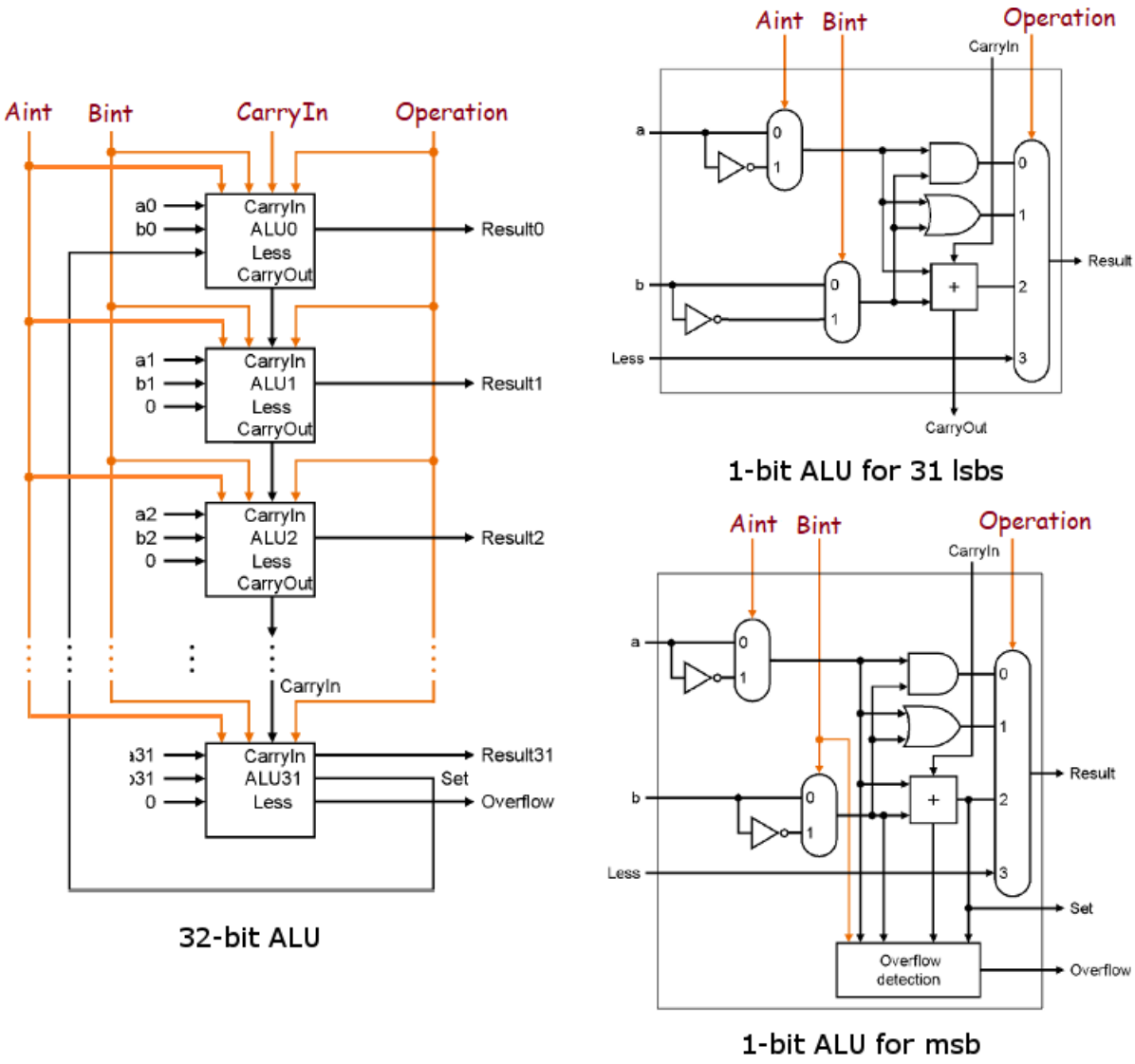
Arithmetic operation	Operand 1	Operand 2	Result (in 4 bits)	Overflow (True/False)
(1.1) Signed Addition	0b0010	0b0100		
(1.2) Unsigned Addition	0b0010	0b0100		
(1.3) Signed Addition	0b0101	0b0110		
(1.4) Unsigned Addition	0b0101	0b0110		
(1.5) Signed Addition	0b1111	0b1011		
(1.6) Signed Addition	0b1001	0b1011		
(1.7) Signed Addition	0b1001	0b0011		
(1.8) Signed Addition	0b0111	0b1000		

คำถาม 2 (10 points)

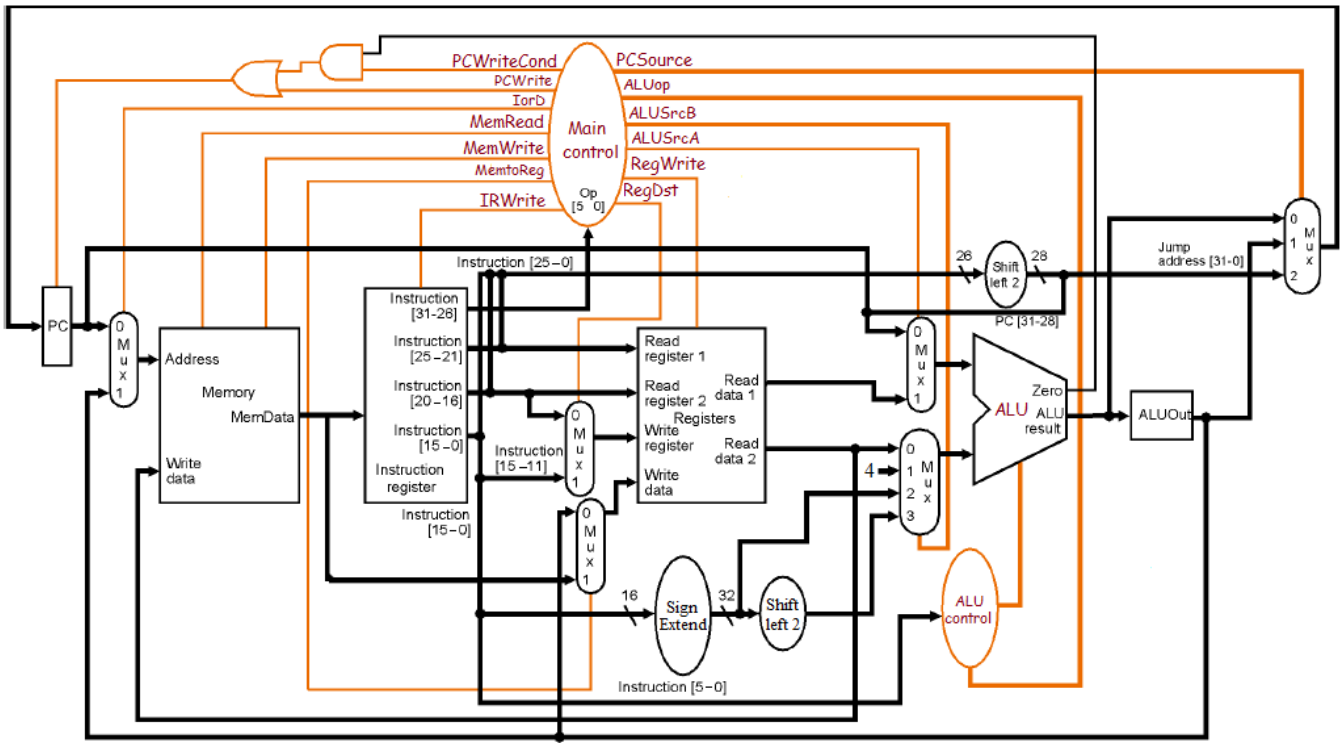
จากรูป 1 เติมตรรกะของวงจร ALU control ให้ถูกต้อง เมื่อแทน ALU0 ถึง ALU30 (31 LSBs) ด้วย 1-bit ALU ที่แสดงในภาพขวาบน และแทน ALU31 (MSB) ด้วย 1-bit ALU ที่แสดงในภาพขวาล่าง. เติมสัญญาณควบคุม (Ainv, Binv, CarryIn, และ Operation) ให้สมบูรณ์ตามการทำงานที่ระบุในคอลัมน์ Goal ของตาราง. [สัญญาณควบคุม: ตอบเป็นฐานสอง; ใช้สัญญลักษณ์ x แทน “don’t care”]

ตารางที่ 1: ตารางตรรกะสัญญาณควบคุม ALU

Input				Output			
ALUOp (2 bits)	Funct (6 bits)		Goal	Ainv	Binv	CarryIn	Operation (2 bits)
00	xxxxxx	(2.1)	add: A + B				
01	xxxxxx	(2.2)	subtract: A - B				
10	100000	(2.3)	add: A + B				
10	100010	(2.4)	subtract: A - B				
10	100100	(2.5)	AND: A & B				
10	100101	(2.6)	OR: A | B				
10	100111	(2.7)	NOR: ~(A | B)				
10	101010	(2.8)	slt: A < B?1:0				



รูปที่ 1: ALU Internal Structure



รูปที่ 2: Multicycle Machine

คำถาม 3 (10 points)

จากแผนภาพของ multi-cycle machine ในรูป 2 เติมสัญญาณควบคุมในตาราง 3 ให้สมบูรณ์ (เติมเองในกระดาษ ไม่มีคะแนน). ดู MIPS Ref. ประกอบ. ALU control มีการทำงานตามระบุในตาราง 1. ALU ต่อกับ ALU control และมีโครงสร้างภายในตามรูป 1.

หลังจากเติมตาราง 3 สมบูรณ์แล้ว เติมค่าสัญญาณควบคุมหลัก (main control) ในตาราง 4 ให้สมบูรณ์ (กำหนดให้ อุปกรณ์ทุกตัวเป็น active high; ดูตารางที่ 1 ประกอบสำหรับ ALUOp) [สัญญาณควบคุม: ตอบเป็นฐานสอง; เพื่อความสะดวกในการตรวจ และลดการสับสน ให้ใช้สัญญลักษณ์ 0 (logic low) แทน “don’t care” สำหรับสัญญาณควบคุม แต่ opcode xxxxxx แทนสถานะ “don’t care”]

ตารางที่ 3: ตารางสถานะสัญญาณควบคุมสำหรับ 3-state machine; หมายเหตุ สัญญาณควบคุมใน step 1 สำหรับคำสั่ง lw และ sw หายไป; สัญญาณควบคุมใน step 2 สำหรับทุกคำสั่งหายไป; คำอธิบายการทำงานของ step 2 sw หายไป (เติมให้สมบูรณ์)

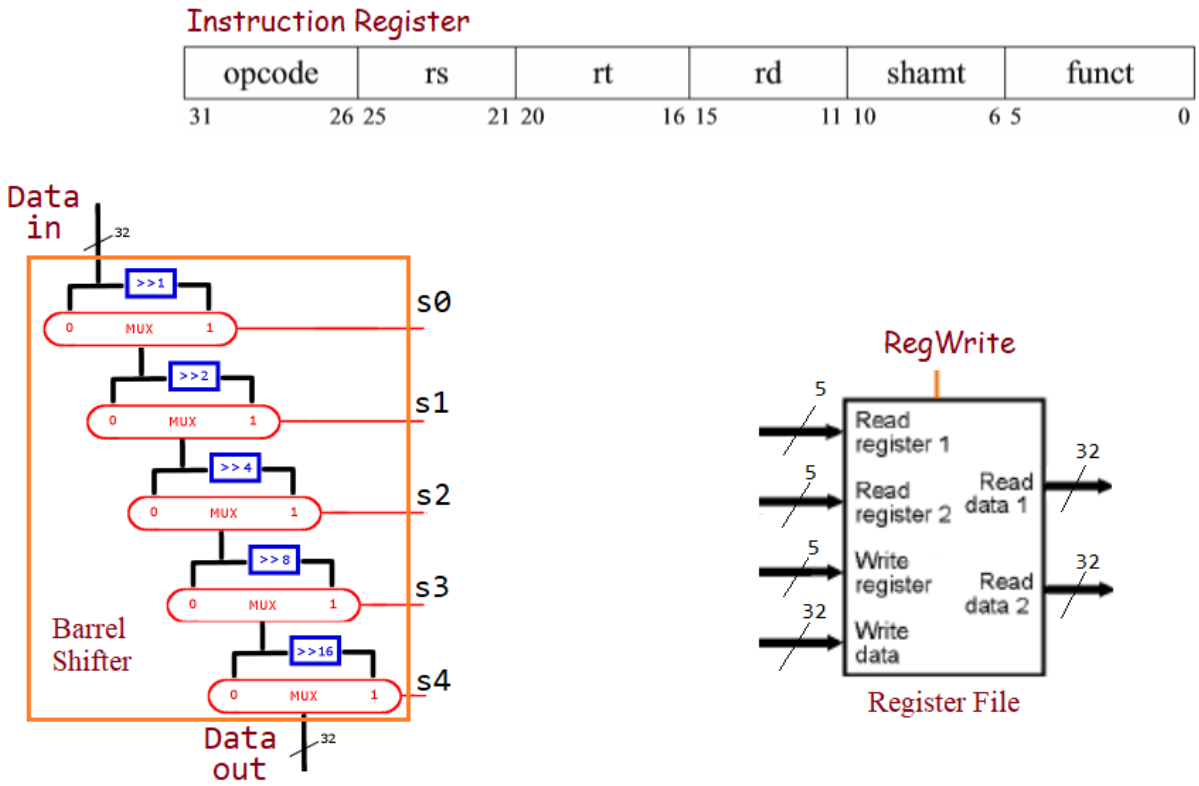
Step	Jump: j	Branch: beq	R-type	Load: lw	Store: sw
0	IR <= Mem[PC]; PC <= PC + 4 IorD=0, MemRead, IRWrite, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSource=00, PCWrite				
1	PC <= {PC[31:28], IR[25:00],00} PCSource=10, PCWrite	ALUOut <= PC + (signext(IR[15:0])<<2) ALUSrcA=0, ALUSrcB=11, ALUOp=00	ALUOut <= Reg[IR[25:21]] op Reg[IR[20:16]] ALUSrcA=1, ALUSrcB=00, ALUOp=10	ALUOut <= Reg[IR[25:21]] + signext(IR[15:0]) ?	
2		if(A==B) PC <= ALUOut ?	Reg[IR[15:11]]<= ALUOut ?	Reg[IR[20:16]]<=Mem[ALUOut] ?	?????????? ?

ตารางที่ 4: ตารางสถานะสัญญาณควบคุมสำหรับ 3-state machine

State	Opcode	PCWrite	PCWriteCond	MemWrite	MemRead	IRWrite	RegWrite	PCSource	IorD	MemtoReg	RegDst	ALUOp	ALUSrcA	ALUSrcB	Next state
0	XXXXXX														
1	000000														
1	000010														
1	000100														
1	100011														
1	101011														
2	000000														
2	000100														
2	100011														
2	101011														

คำถาม 4 (10 points)

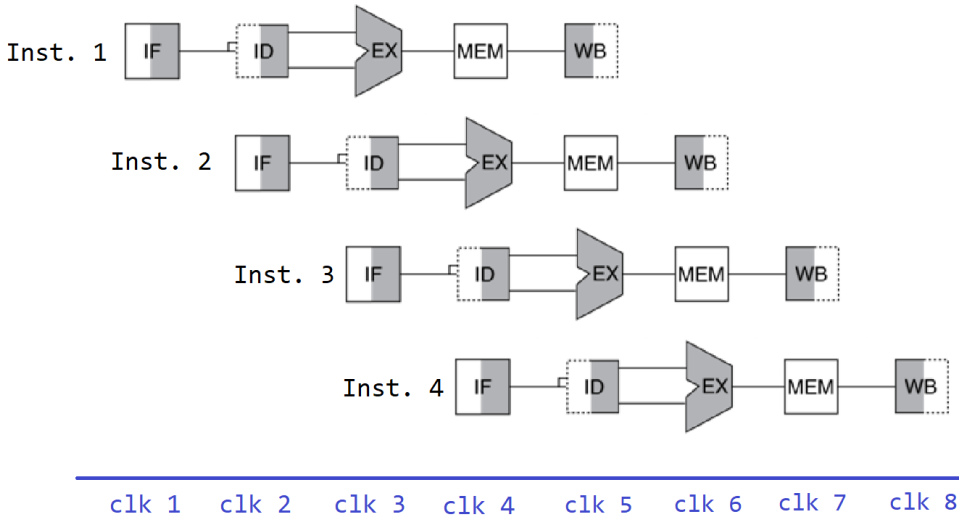
จาก diagram ในรูป 3 และคำอธิบายคำสั่ง srl ใน MIPS Ref. เติม datapath ใน diagram สำหรับคำสั่ง srl. เขียนสายสัญญาณเชื่อมต่ออุปกรณ์ทั้งสาม (IR, Barrel Shifter, และ Register File). เขียนสายสัญญาณให้ชัดเจน พร้อมระบุบิตในกรณีที่คลุมเครือ. เขียนคำตอบในกระดาษ



รูปที่ 3: Diagram without connection

คำถาม 5 (10 points)

สำหรับ 5-stage pipeline: IF (instruction fetch), ID (instruction decode), EX (ALU execution), MEM (memory access), และ WB (write back to register) ที่ใช้ clock rate 1GHz (clock period $= 1\mu s$) ตอบคำถามต่อไปนี้ [ใช้ terminology ให้ถูกต้อง]



รูปที่ 4: 5-stage pipeline

(5.1) ถ้าสามารถทำ pipeline ได้สมบูรณ์

- หนึ่งคำสั่งใช้เวลาทำงานเท่าไร
- 10 คำสั่งใช้เวลาทำงานเท่าไร
- $1,000,000$ คำสั่งใช้เวลาทำงานเท่าไร

(5.2) จากรูป 4 สังเกตุว่าที่ clk 4 ถ้าคำสั่งที่ 1 อ่านข้อมูล จาก memory และคำสั่งที่ 4 อ่านคำสั่ง จาก memory. ถ้า memory ที่ทั้งสองคำสั่งต้องการอ่าน เป็นอุปกรณ์เดียวกัน จะมีปัญหาการแย่งทรัพยากรกัน ปัญหาแบบนี้เรียกว่าอะไร

(5.3) จากรูป 4 ถ้าคำสั่งที่ 1 เป็น beq $s0, $s1, 0x1200 ซึ่ง ถ้าค่า $s0 และ $s1 เท่ากัน คำสั่งต่อไปจะเป็นคำสั่งของแอดเดรสที่ PC+4 + 0x1200. แต่หากถ้าค่า $s0 และ $s1 ไม่เท่ากัน คำสั่งต่อไป ก็คือคำสั่งที่แอดเดรสต่อจากคำสั่งที่ 1 (คำสั่งที่แอดเดรส PC+4). แต่คำสั่งที่ 2 จะถูกอ่านเข้ามาทำที่ clk 2 แต่กว่าที่ผลการเปรียบเทียบจะเสร็จ (ทำใน EX stage ของคำสั่ง 1) ก็ต้องหลัง clk 3. ดังนั้นที่ clk 2 อาจจะอ่านคำสั่งที่ 2 มาผิดคำสั่ง. ปัญหาในลักษณะนี้ เรียกว่าอะไร

(5.4) จากรูป 4 ถ้าคำสั่งที่ 1 เป็น addi $s0, $zero, 0x1200 และคำสั่งที่ 2 เป็น ori $s1, $s0, 0x0A00 สังเกตุว่า คำสั่งที่ 2 จะใช้ค่าใน $s0 เพื่อคำนวณ (EX stage) ใน clk 4 แต่ผลล่าสุดของ $s0 ที่ได้จากการทำคำสั่งที่ 1 จะยังไม่ถูกเขียนเข้ารีจิสเตอร์จนกว่าจะถึง clk 5. ดังนั้นหาก คำสั่งที่ 2 คำนวณ โดยใช้ค่าในรีจิสเตอร์ ที่ไม่ใช่ค่าล่าสุดที่เพิ่งคำนวณได้ จะทำให้เกิดปัญหาทำงานไม่ถูกต้องได้ ปัญหาแบบนี้เรียกว่าอะไร

(5.5) สังเกตุว่า ปัญหาที่อภิปรายในข้อ 5.4 ค่าล่าสุดของ $s0 นั้นมีพร้อมแล้วที่ clk 4 (เพราะได้จาก EX stage ของคำสั่งที่ 1 ที่ทำใน clk 3) เพียงค่าล่าสุดนี้ จะยังไม่ได้ถูกเขียนเข้าไปในรีจิสเตอร์. ดังนั้นวิธีแก้ปัญหาลักษณะนี้ ก็คือ เพิ่มเส้นทางของข้อมูลอีกเส้นทาง เพื่อลัดการนำข้อมูลไปใช้เลย โดยไม่ต้องรอ (ไม่ต้องรอจนค่านี้ถูกเขียนลงรีจิสเตอร์เสร็จ แล้วค่อยอ่านค่านี้กลับออกมาอีก). วิธีการเช่นนี้ เรียกว่าอะไร

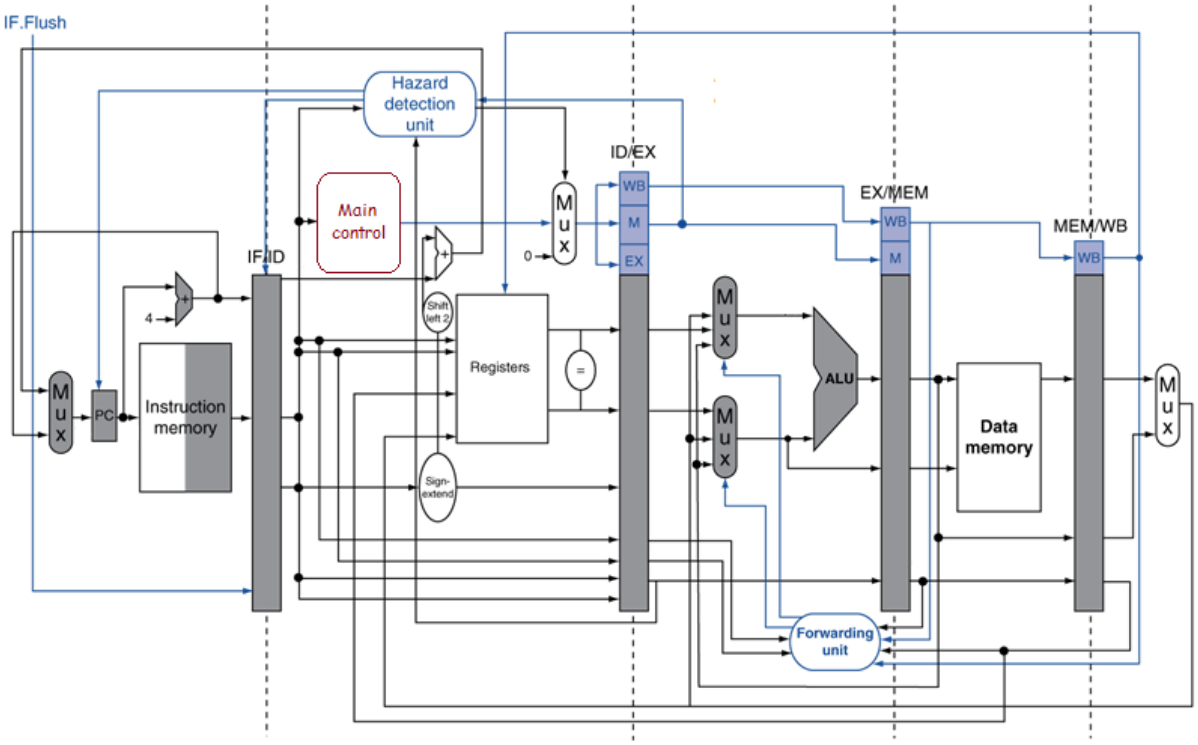
(5.6) จากรูป 4 ถ้าคำสั่งที่ 1 เป็น lw $s0, 0($s5) และคำสั่งที่ 2 เป็น addi $s1, $s0, 1 สังเกตุว่า แม้สถานการณ์นี้จะดูคล้ายๆ สถานการณ์ในข้อ 5.4 แต่สถานการณ์ในข้อ 5.4 สามารถใช้วิธีการในข้อ 5.5 ช่วยได้. แต่เนื่องจาก คำสั่งที่ 2 ต้องการค่าล่าสุดของ $s0 ที่ clk 4 แต่ค่าล่าสุดของ $s0 ของคำสั่งที่ 1 จะยังไม่ถูกอ่าน ออกจากหน่วยความจำ จนกว่าจะจบ clk 4 ซึ่งไม่ทันเวลาที่ EX stage ของคำสั่งที่ 2 ต้องการ. ลักษณะปัญหาในสถานการณ์แบบนี้ เรียกว่าอะไร

(5.7) สำหรับสถานการณ์ที่อภิปรายในข้อ 5.6 อาจช่วยบรรเทาได้โดยพักการทำคำสั่งที่ 2 ก่อนจนกว่าข้อมูลที่ต้องการจะพร้อม วิธีการพักคำสั่งนี้ ช่วยให้ได้คำตอบที่ถูกต้อง แต่ทำให้ประสิทธิภาพ ของ pipeline ลดลง (พัก 1 จังหวะ เท่ากับ pipeline จะช้าลง 1 clock) วิธีการพักการทำคำสั่งเช่นนี้ เรียกว่าอะไร

(5.8) กลไกที่คอมไพเลอร์ สามารถช่วยการทำงานของ pipeline ได้ โดยการจัดเรียงลำดับคำสั่งให้ตรรกะการทำงานและผลลัพธ์เหมือนเดิม แต่สามารถลดการหยุดพักการทำคำสั่ง (ที่อธิบายในข้อ 5.7) กลไกนี้เรียกว่าอะไร

(5.9) Branch prediction เป็นวิธีการเพื่อบรรเทาปัญหาเรื่องใด

(5.10) Memory hierarchy สามารถช่วย pipeline ได้ เทคนิคนี้สามารถช่วยลดปัญหาของ pipeline ในเรื่องใด



รูปที่ 5: Pipeline machine

คำถาม 6 (10 points)

จากรูป 5 สังเกตุว่า โครงสร้างของ pipeline machine ซับซ้อนกว่า multi-cycle machine มาก และ pipeline machine แยกหน่วยความจำออกเป็น instruction memory และ data memory. Hazard detection unit ทำหน้าที่ตรวจสอบ และจัดการปัญหา ที่อาจเกิดใน pipeline. Forwarding unit ทำหน้าที่ควบคุม เส้นทางลัดของข้อมูล ซึ่งเป็นกลไกสำคัญของ pipeline. การที่ pipeline machine สามารถ ทำงานได้หลายๆ stages พร้อมๆกันนั้น เพราะในโครงสร้างของ pipeline machine ใช้ stage registers (IF/ID, ID/EX, EX/MEM, และ MEM/WB ในรูป). สังเกตุว่า หน่วย control ซึ่งเป็นหน่วยที่จ่ายสัญญาณควบคุมอุปกรณ์ต่างๆ มีอยู่ชุดเดียว อยู่ที่ ID stage (ระหว่าง IF/ID register และ ID/EX register ในรูป). อภิปรายสั้นๆว่า กลไกอะไรที่ทำให้ stages อื่นๆ หลัง ID stage ได้รับสัญญาณควบคุมที่ถูกต้อง. เขียนคำตอบในกระดาษ